

Scapia Market Intelligence — Architecture Brief

What we have built · what we need to host it

Scapia Market Intelligence is an LLM-maintained wiki tracking competitors, regulators, market signals, and partners. Curators feed raw sources; an LLM filter scores them; approved sources are written as structured wiki pages by Claude. Leaders ask plain-English questions and get a sourced "Brief" answer. The wiki is maintained by the LLM — not by humans.

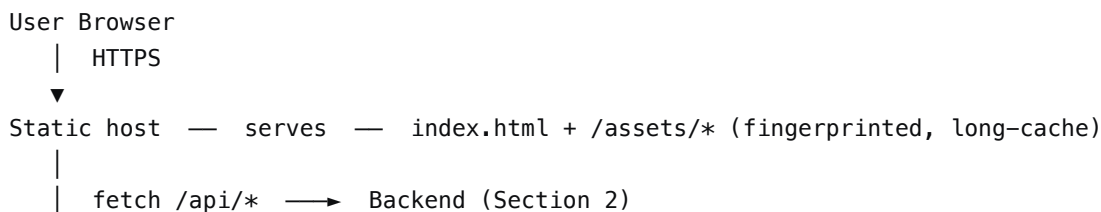
1 Frontend

WHAT WE HAVE

- React + Vite single-page app (`frontend-react/`)
- 9 routes — 5 leader surfaces, 4 curator/legacy
- Production build = static files in `dist/` (~228 KB initial load gzipped)
- Lazy-loaded curator + heavy leader chunks
- Wired to read backend URL from environment variable

WHAT WE NEED

- Static file hosting reachable by users (with HTTPS + a domain)
- SPA routing — any unknown path falls through to `index.html` (config file provided)
- CDN-style caching for fingerprinted `/assets/*`; `index.html` stays uncached so deploys take effect immediately
- Build pipeline that runs `npm run build` on push and publishes `dist/`
- Environment variable injected at build: `VITE_API_URL` pointing at the backend



2 Backend

2.A · AUTH & ROLES

WHAT WE HAVE

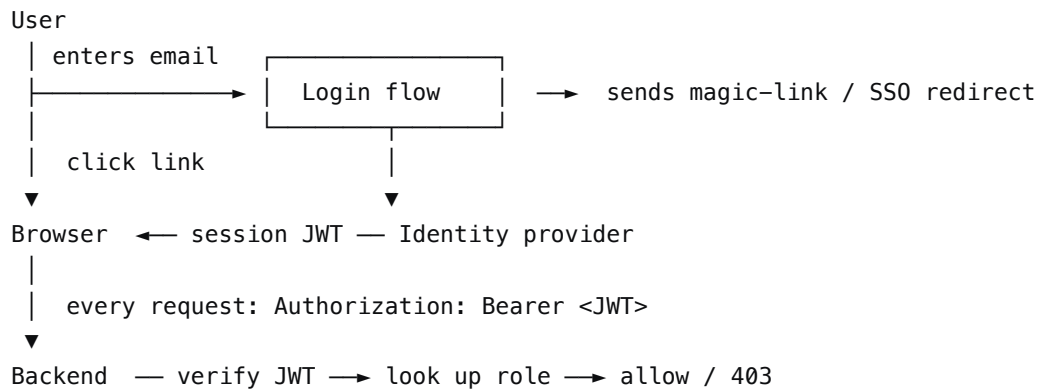
- **No auth screen, no login, no users.** The app runs on localhost — anyone with the dev URL has full access.

WHAT WE NEED

- Login screen + email-based identity (passwordless magic-link is the assumed flow; SSO is acceptable if the org has one)

- No role separation in code; the curator surfaces (`/curator` , `/submit`) are reachable by typing the URL.
- No JWT, no session, no email flow.

- Two roles: **leader** (read + ask questions) and **curator** (read + submit + review)
- Role stored server-side and checked on every backend request (frontend role-gating is convenience only)
- **Curator-only routes:** any `/submit` , `/curator/*` path and the corresponding write endpoints must return `403` for leaders
- Identity surface: a way to invite users and assign their role (no admin UI needed — direct DB row is fine for <10 users)
- A separate header-based secret (`X-Internal-Secret`) for cron-triggered endpoints — not exposed to users



2.B · AGENT LAYER (INGEST + QUERY)

WHAT WE HAVE

- Single Python file `server.py` (~1,400 lines) using stdlib `http.server` — fine locally, not production-grade
- Filesystem reads/writes for wiki + raw + state
- 10 HTTP endpoints (table below)
- 5 LLM-driven roles in the pipeline (Haiku for filters/classifier, Sonnet for writers)
- Synchronous: a submit blocks ~30–60s while the agent runs

WHAT WE NEED

- **FastAPI rewrite** — async event loop (needed for streaming answers), input validation, OpenAPI docs, middleware for auth and CORS, graceful shutdown
- Migration to **Claude Agent SDK** — proper tool-use loop, retries, structured tool errors
- Background-task runner for ingest (submit returns immediately; the agent runs async)
- Server-sent events (SSE) streaming for query answers (first token under 1s)
- Restart recovery — on boot, mark any jobs left `running` as `failed`

Current API endpoints (in `server.py`)

METHOD	PATH	PURPOSE	USED BY
GET	/api/pages	List wiki pages with frontmatter	Vault, Intel Cards
GET	/api/page?path=...	Single page body + meta	Page panel
GET	/api/feed	Source-summary feed items (time-ordered)	Homepage "What's New"
GET	/api/raw / /api/raw/content	List + read raw source files	Curator view
GET	/api/state	Pending queue, rejections, health counts	Curator page
POST	/api/query	Ask question, get Brief answer (blocking)	Intelligence page
POST	/api/submit	Submit raw source → filter → maybe ingest	Submit form
POST	/api/queue/approve	Curator approves a Layer-2 borderline card	Curator page
POST	/api/queue/reject	Curator rejects + deletes raw file	Curator page

Additional endpoints we will need

METHOD	PATH	PURPOSE
GET	/api/me	Current user, role, today's usage counts
GET	/api/jobs	Async ingest jobs list (curator)
GET	/api/jobs/{id}	Single job with full agent trace
POST	/api/layer2/{id}/decide	Curator resolves a borderline card
GET	/api/ingest-log	Append-only operation log
POST	/internal/send-digest	Daily digest email (scheduled)
POST	/internal/snapshot	Nightly DB → markdown export (scheduled)
POST	/internal/keepalive	Health ping (scheduled)

LLM roles in the pipeline

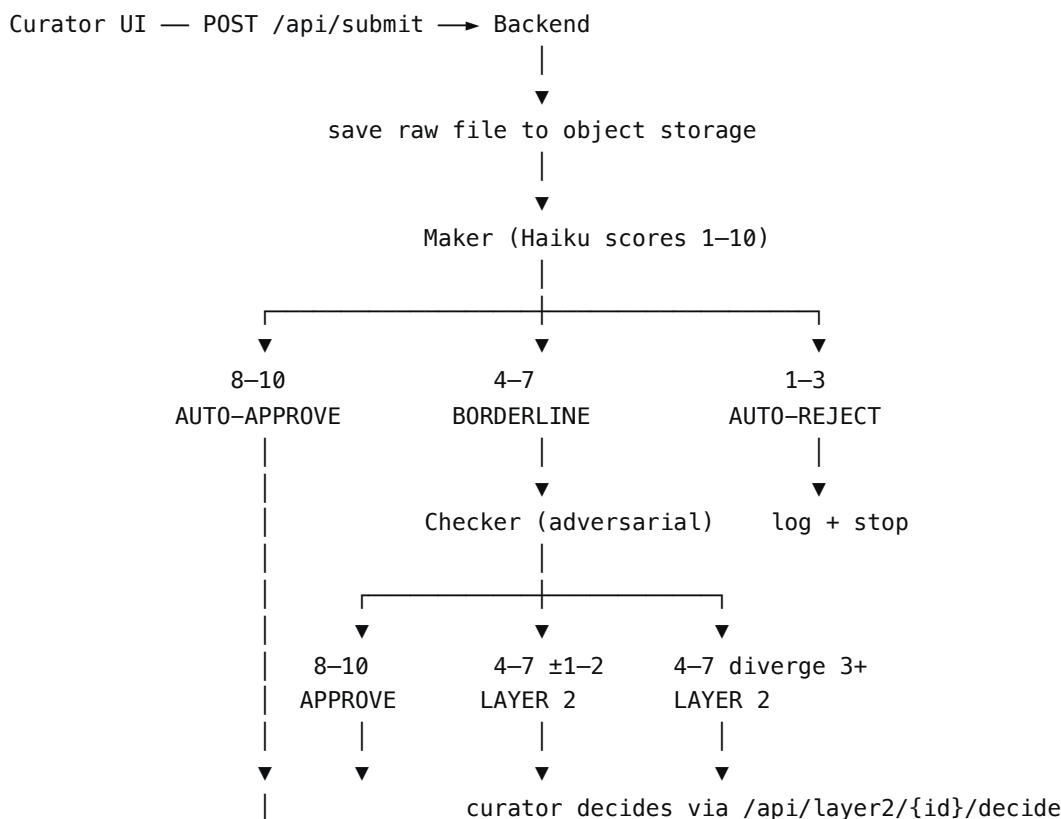
ROLE	MODEL	WHERE IT RUNS	PURPOSE
Intent Classifier	Haiku	Every /api/query	Routes question to <code>brief_query</code> / <code>clarification</code> / <code>out_of_scope</code> / <code>meta</code> / <code>greeting</code> — saves LLM cost on small talk
Maker (filter)	Haiku	Every /api/submit	Scores 1–10 across relevance · duplication · incremental value

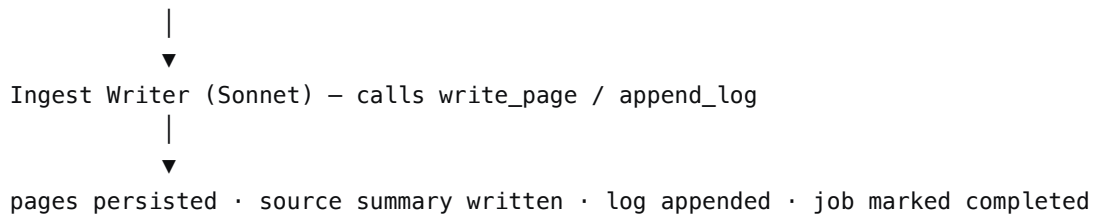
Checker (adversarial filter)	Haiku	Only if Maker scored 4–7	Adversarial second-pass; tries to <i>refute</i> the source
Ingest Writer	Sonnet	When filter approves	Writes / updates structured wiki pages, follows the strict <code>CLAUDE.md</code> schema
Query Responder	Sonnet	When intent = <code>brief_query</code>	Composes the Brief-format answer with inline citations; streams via SSE

Tools the agent will call (when migrated to the Agent SDK)

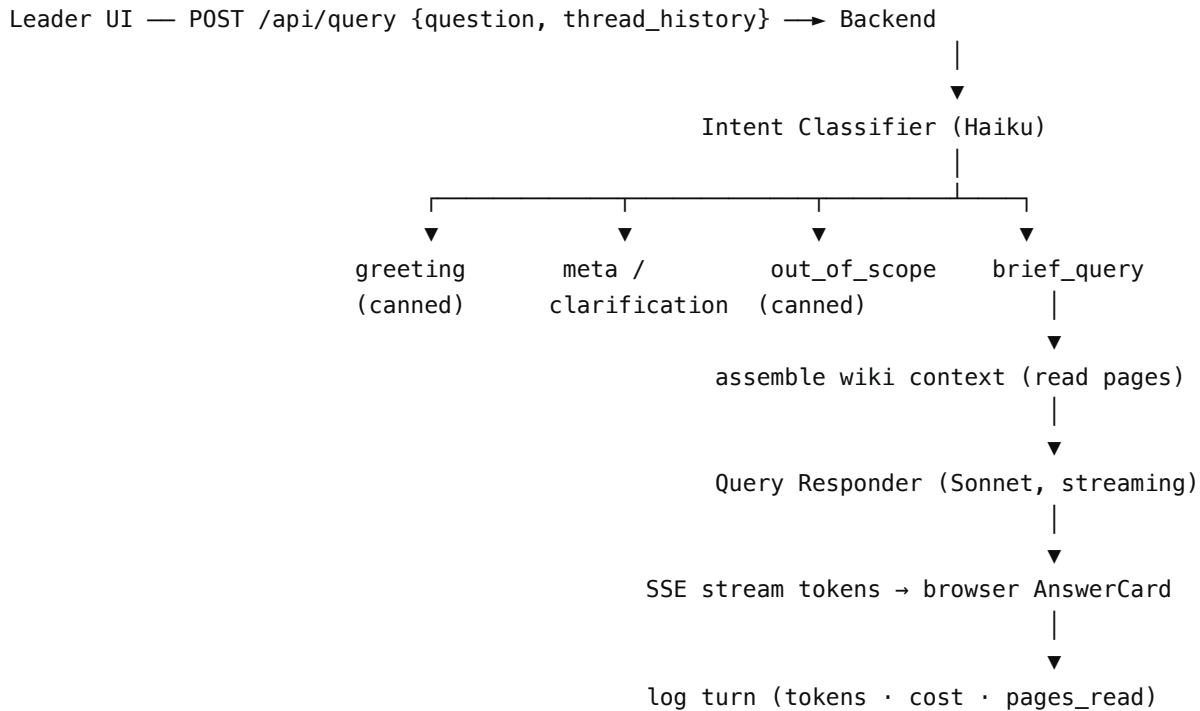
TOOL	PURPOSE
<code>read_raw(path)</code>	Read a raw source from object storage
<code>list_pages(folder?, type?)</code>	List wiki pages with summary fields
<code>read_page(path)</code>	Read a wiki page's body + frontmatter
<code>write_page(path, frontmatter, body)</code>	Validate, parse, upsert (hard contract — rejects if mandatory section missing)
<code>upload_raw(path, content)</code>	Capture verbal context as a raw source
<code>append_log(operation, description)</code>	Add an entry to the append-only log
<code>mark_layer2(...)</code>	Halt workflow, surface a borderline review card

Ingest flow





Query flow



Cost & rate guardrails (every LLM call goes through them)

#	LAYER	WHERE	WHAT IT BLOCKS
1	Circuit breaker	In-memory, per backend container	Halts all LLM calls if upstream errors spike (5 errors / 60s)
2	Per-minute rate limit	Per user	Throttles burst submissions
3	Concurrent ingest semaphore	DB count check	Caps simultaneous agent loops (queues the rest)
4	Per-user daily caps	SUM over today	Queries · input tokens · output tokens
5	Cost velocity tripwire	SUM over last hour	Catches runaway loops
6	Monthly hard kill	SUM over month	Absolute budget ceiling
7	Per-ingest token cap	Between agent iterations	Stops one job hogging the daily envelope
8	Iteration ceiling + stuck-loop detect	Between agent iterations	30 iterations max; same-tool-same-args ×3 = abort

2.C · DATABASE

What we have today — filesystem

```
v1-LLM-wiki/
├── wiki/                ← LLM-written knowledge base (markdown files)
│   ├── index.md         ← master catalog of all pages (LLM maintains)
│   ├── log.md           ← append-only event log (LLM maintains)
│   ├── scapia.md        ← reference baseline about Scapia itself
│   ├── entities/        ← one file per competitor (Axis Atlas, CRED, HDFC...)
│   ├── partners/        ← Federal Bank, Visa/MC/RuPay, OTAs
│   ├── regulatory/      ← RBI circulars, DPDP, NPCI rules
│   ├── events/          ← discrete market moments (launches, devaluations)
│   ├── market-signals/  ← macro trends
│   ├── concepts/        ← shared mechanics (FX markup, MDR...)
│   ├── comparisons/     ← head-to-head, on demand
│   └── sources/         ← one summary file per ingested source
├── raw/                 ← immutable ingested source files (audit trail)
│   ├── competitive/     ← Reddit, news, analyst notes
│   ├── regulatory/      ← RBI circulars, MeitY notifications
│   ├── market/          ← industry reports, traffic data
│   └── ambiguous/       ← sources spanning multiple treatments
├── state.json           ← pending Layer-2 queue, rejection log, health counts
└── browser localStorage ← per-user query thread history, read state (orange dots)
```

What we need — PostgreSQL with object storage for raw files

Markdown stays markdown (stored as a text column in the page row). Frontmatter is a JSON column so flexible per-page-type fields work. Hot-filter fields (folder, signal, posture, ...) are denormalised into real indexed columns for sub-millisecond filtering. The Intel section is parsed at write time and stored as JSON so the frontend reads it directly.

pages — every wiki page (entities, regulatory, events, partners, market-signals, concepts, comparisons, sources)

Columns: path PK · folder · slug · type · page_types text[] · domains text[] · travel_categories text[] · headline · signal · vitals jsonb · posture · frontmatter jsonb · body text · intel_section jsonb · last_updated · sources_count · created_at · updated_at

Indexes: folder · signal · posture · updated_at DESC · GIN(page_types) · GIN(domains)

sources — the live feed (auto-derived from pages where folder='sources' via DB trigger)

Columns: slug PK · headline · source_title · source_file · ingested_at · touches text[] · primary_entity · signal · page_types text[] · domains text[] · filter_score · filter_band

ingest_jobs — async job state machine + full agent trace

Columns: id uuid PK · raw_path · submitted_by → users · status · agent_trace jsonb · result jsonb · error · created_at · updated_at · completed_at

Status values: `queued` · `running` · `completed` · `layer2` · `rejected` · `failed` · `cost_capped`

ingest_log — append-only audit log of every wiki operation

Columns: id bigserial PK · occurred_at · operation · description

Operations: `ingest` · `query` · `lint` · `update` · `create` · `filter-reject` · `filter-layer2`

query_threads — groups a user's follow-up questions into a single chat (moves chat history from browser localStorage to the DB, cross-device)

Columns: id uuid PK · user_id → users · title · created_at · updated_at

Index: (user_id, updated_at DESC)

query_log — every Q&A turn (doubles as the per-query cost record)

Columns: id bigserial PK · thread_id → query_threads · position int · user_id → users · question text · answer text · pages_read text[] · input_tokens · output_tokens · cost_usd · created_at

Indexes: (thread_id, position) · (user_id, created_at DESC)

ingest_cost_log — token + cost record per ingest job

Columns: job_id → ingest_jobs · user_id → users · input_tokens · output_tokens · cost_usd · created_at

user_profiles — role for every authenticated user

Columns: user_id PK → auth users · name · role (leader | curator) · created_at

subscribers & **digest_log** — daily email digest

subscribers : id · email · name · role · paused · created_at

digest_log : id · sent_at · subscriber_email · source_slugs text[] · provider_message_id (idempotency)

Views (read-only aggregations the backend queries)

VIEW	WHAT IT RETURNS	USED BY
user_daily_usage	Per-user counts + sums for today (queries · input tokens · output tokens) — aggregates query_log + ingest_cost_log	Cost cap pre-flight; sidebar usage display
monthly_spend	Total USD across all users for the current month	Monthly hard-kill check
cost_velocity_hourly	SUM(cost_usd) WHERE created_at > now() – 1 hour	Cost velocity tripwire

Triggers

- **pages → sources auto-derive:** on INSERT/UPDATE of a pages row where folder='sources' , derive the corresponding sources row from the page's frontmatter. The agent calls one tool (write_page); the trigger keeps the feed table consistent — zero risk of drift.

Row-Level Security (defense in depth)

- query_threads + query_log : USING (user_id = current_user_id()) — a user can **never** read another user's chats even if the backend has a bug.
- All other tables: read open to authenticated users; writes go through the backend's service-role key only.

Object storage (not in the DB)

- One private bucket: raw-sources/ — every raw markdown/PDF/text source the curator submits
- Paths mirror today's raw/competitive/... , raw/regulatory/... structure

- Backend reads via signed URLs; the agent's `read_raw` / `upload_raw` tools touch this bucket

Cost & usage tracking — what gets stored on every LLM call

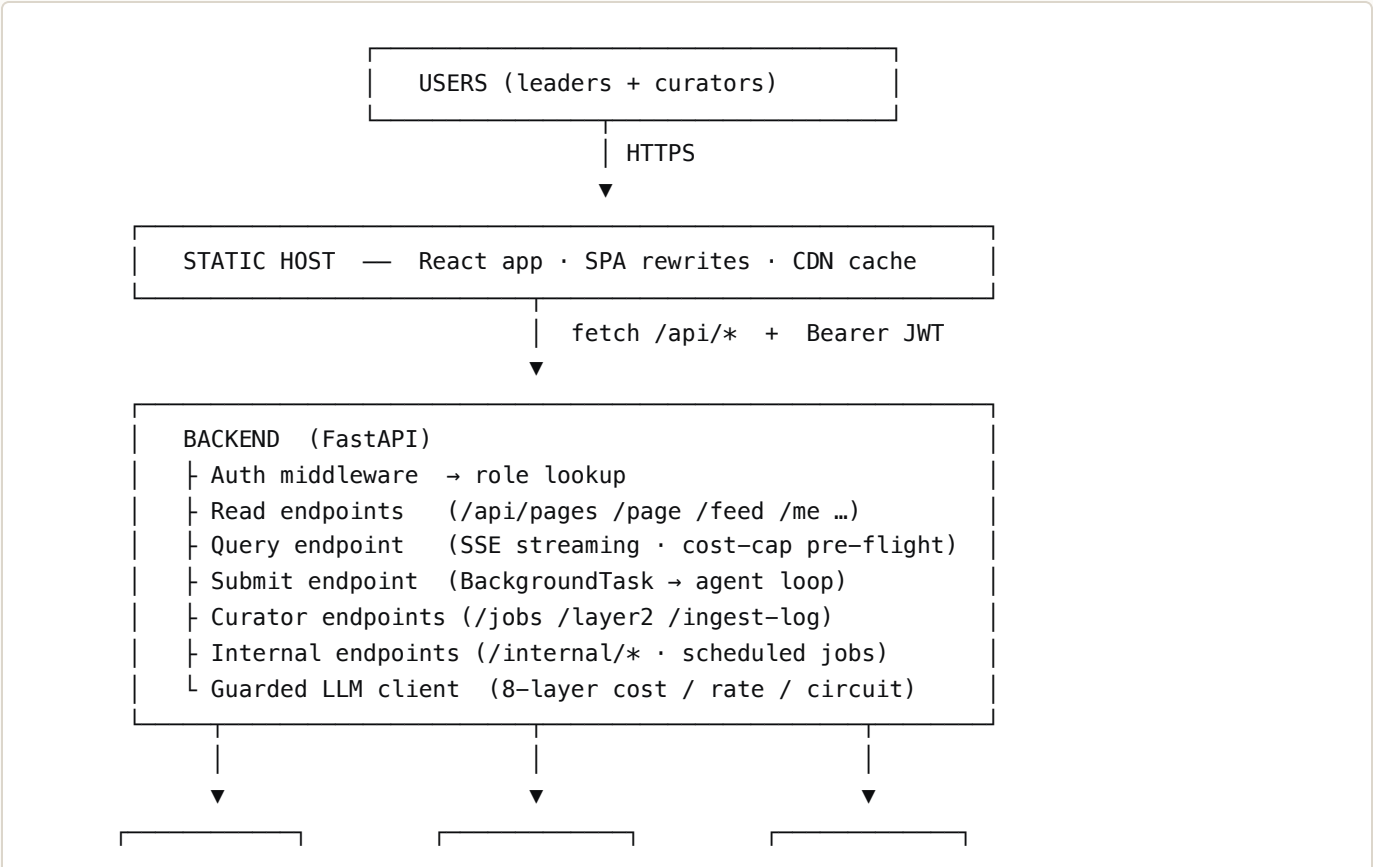
SURFACE	STORED WHERE	FIELDS
Every query turn	<code>query_log</code> (one row per turn)	question · answer · pages_read · input_tokens · output_tokens · cost_usd
Every ingest job	<code>ingest_cost_log</code> (one row per job)	job_id · input_tokens · output_tokens · cost_usd
Filter calls (Maker · Checker · Intent)	Folded into the parent job/query row (input/output tokens aggregated)	(no separate table — keeps the cost surface simple)

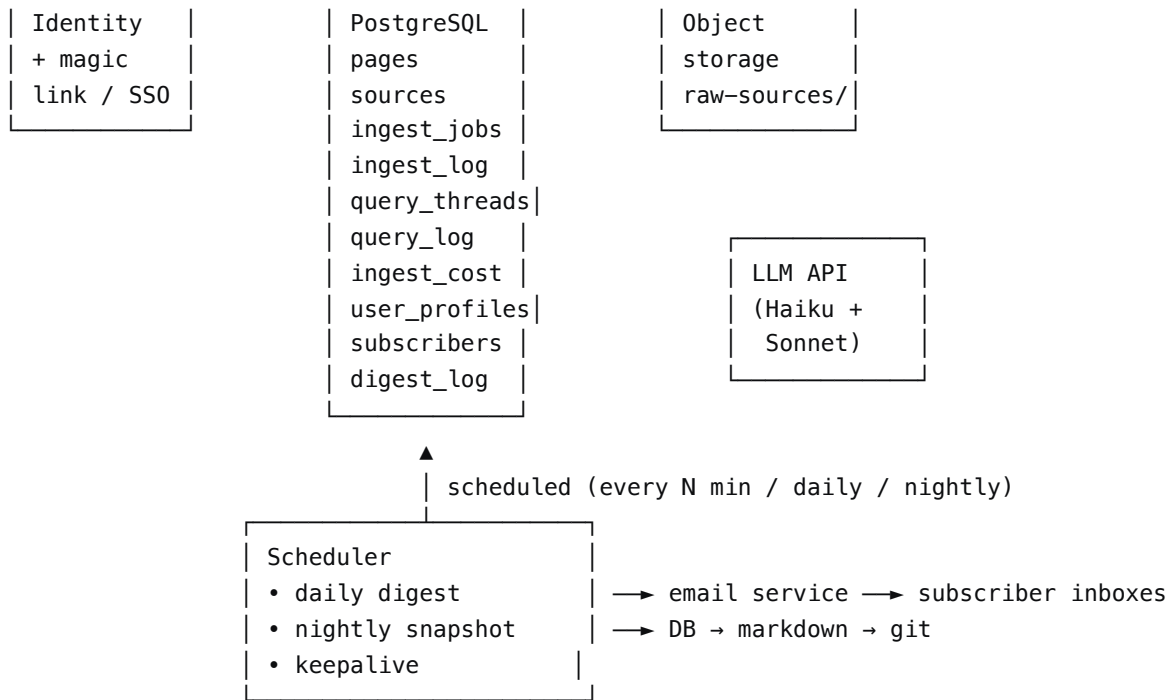
Daily caps and monthly limits are **computed from these two tables** — no separate counters to keep in sync. The cost-cap pre-flight check is a single indexed SUM under 5 ms.

Query history saving — explicit

- Today: stored only in the user's **browser localStorage** — lost on browser switch / clear cache.
- Production: stored in `query_threads` + `query_log` scoped to `user_id` . Cross-device, durable, and the same rows power cost-cap aggregation.
- RLS makes one user's chats invisible to every other user at the DB level — not just at the API layer.
- Wiki pages are **never** joined to query tables — strict separation between "outside view" (wiki) and "inside view" (a user's chat).

3 Full system, one picture





What we will need from the infra side, in plain English

LAYER	WHAT WE NEED (NO SPECIFICS)
Frontend	A place to host static files with HTTPS, a custom domain, SPA-style routing, and CDN-cached assets. A build step on every push.
Backend	A long-running Python process reachable on HTTPS. Must talk outbound to the LLM API. Async event loop, ~512 MB–2 GB memory, restarts cleanly on deploy.
Database	A managed PostgreSQL with the schema above. Private — only the backend reaches it. Automated daily backups.
Object storage	One private bucket for raw source files. Backend has read/write; nobody else does.
Email	A transactional email sender with a verified Scapia-owned sending domain. Used for magic-link login and daily digest.
Identity / Auth	Magic-link email auth, or SSO if the org has a standard. Sends a JWT the backend can verify.
Secrets	A secure store for API keys, DB password, JWT signing key, internal secret. Backend reads at startup.
Scheduler	A way to call three internal HTTP endpoints on a schedule (daily, nightly, every-N-minutes).
Observability	Logs from the backend captured centrally. Alerts on error spikes, OOM, DB CPU.
CI / Deploy	On every push to <code>main</code> : build frontend → publish; build backend container → restart. Whatever standard pattern the team uses.

Architecture brief · current state vs. hosted state · no vendor-specific recommendations · the repository's `DEPLOYMENT-ARCHITECTURE.md`, `KNOWLEDGE-TRANSFER.md`, and `CLAUDE.md` are the long-form references behind this summary.